System Architecture & Engineering Specification



1. Executive Summary & System Overview

This document defines the technical architecture and software engineering specification for the platform, a full-stack skill discovery, mentorship, and professional networking ecosystem. The architecture connects individual users, industry mentors, corporate entities, and educational institutions within a unified digital environment.

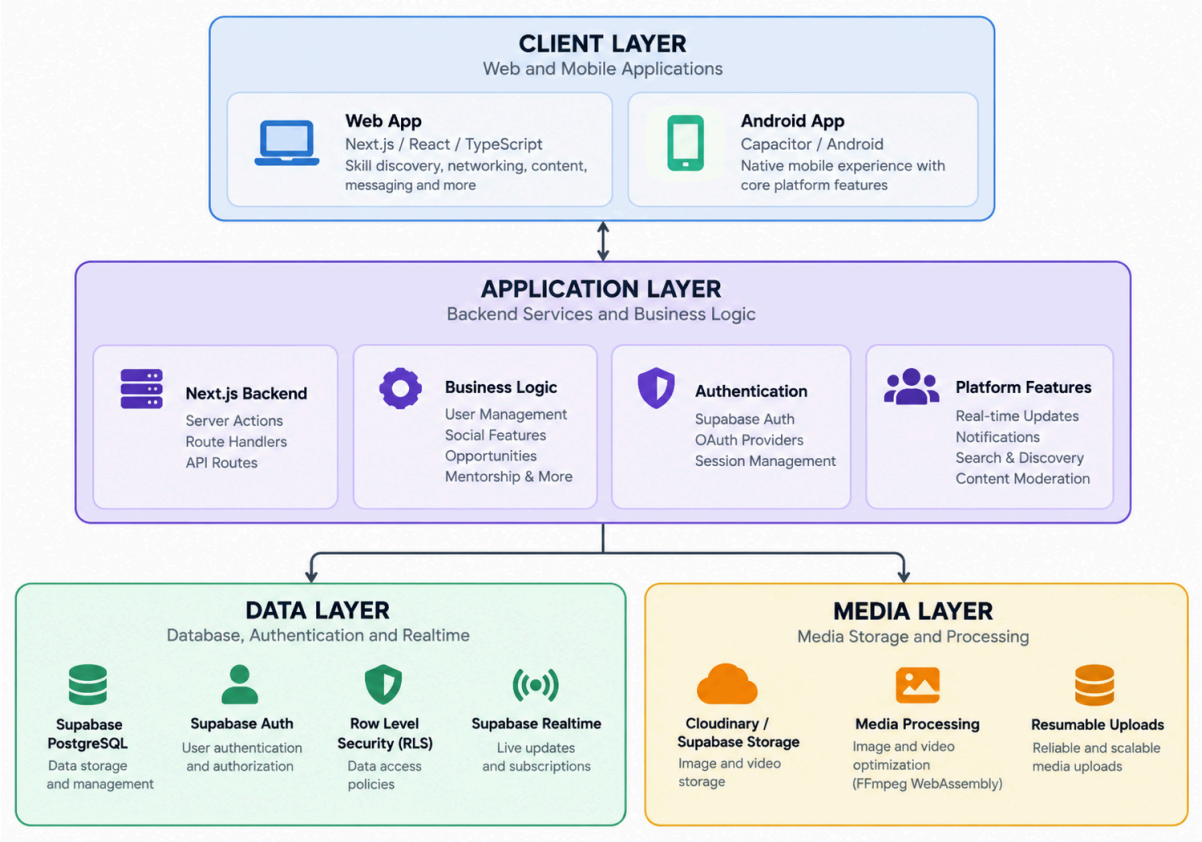
The core web platform is implemented with Next.js, React, and TypeScript, leveraging Supabase for identity management, relational database services, Row Level Security (RLS), and real-time subscription capabilities. Native mobile deployment is enabled via Capacitor for Android targets. Key functional domains include verified talent profiles, skill-based networking, algorithmic feeds, real-time messaging, notification pipelines, opportunity matching, structured mentorship, short-form media (Stories and Reels), gamification, and automated content moderation.

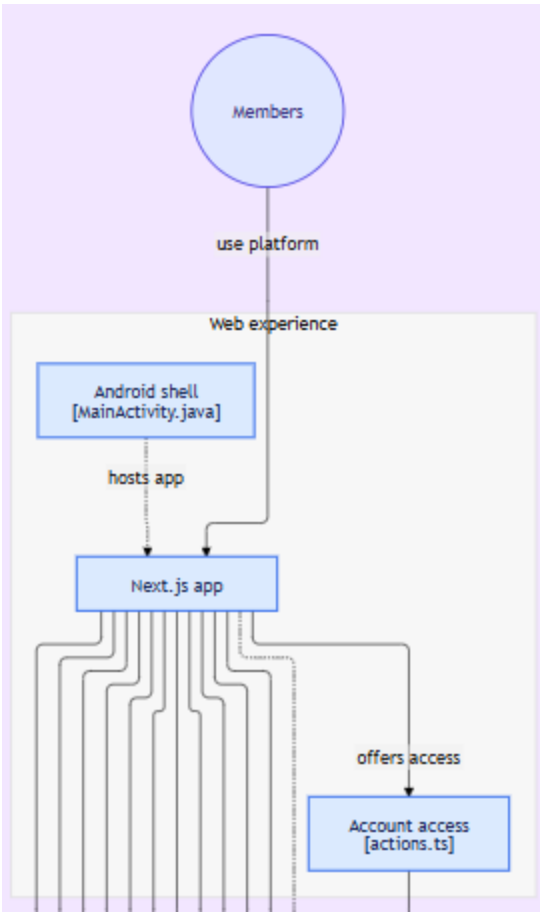
1.1 Technology Stack Matrix

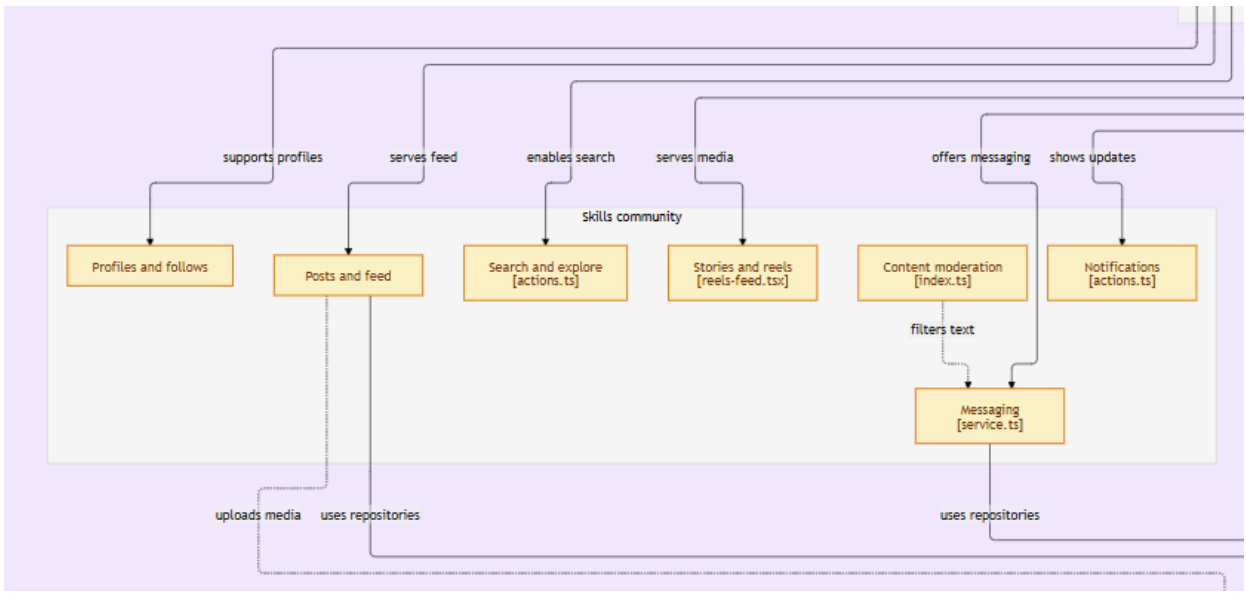
Layer	Technology Choice	Purpose & Architectural Notes
Web Frontend	Next.js 15, React 19, TypeScript	App Router, Server Components, and Server Actions
Styling & UI	Tailwind CSS v4, Framer Motion	Modern styling engine with micro-animations and smooth motion
State & Forms	TanStack Query, Zustand, React Hook Form + Zod	Asynchronous data fetching, global UI state, and schema validation
Backend / DB	PostgreSQL via Supabase	Row Level Security (RLS), Realtime sockets, and Auth integration
Mobile Runtime	Capacitor (Android)	Wraps web interface for native Android delivery
Media Handling	Cloudinary & Supabase Storage	Hybrid image/video asset processing, resizing, and resumable uploads

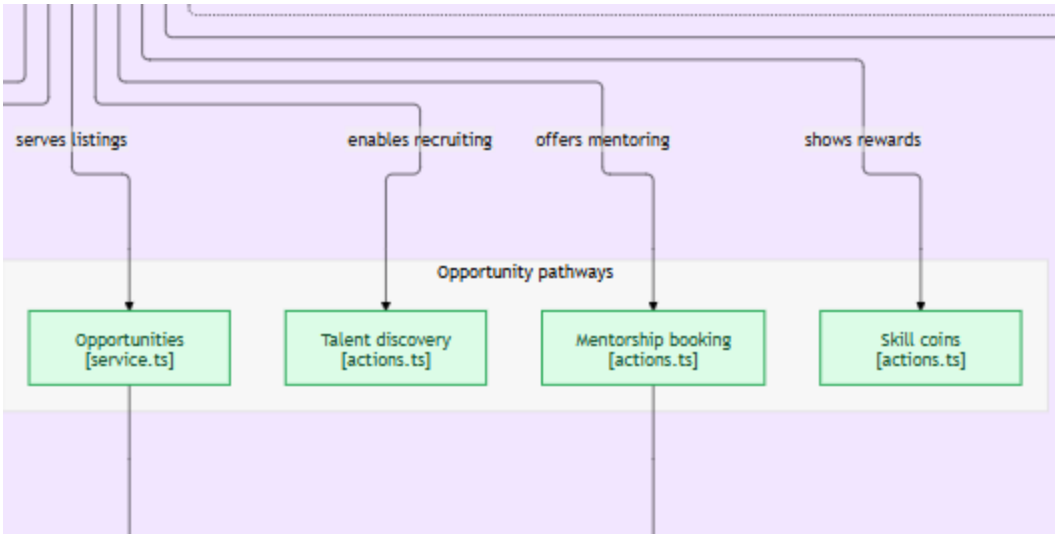
Layer	Technology Choice	Purpose & Architectural Notes
Testing & CI	Vitest, Playwright, GitHub Actions	Unit testing, full E2E automation, and continuous verification

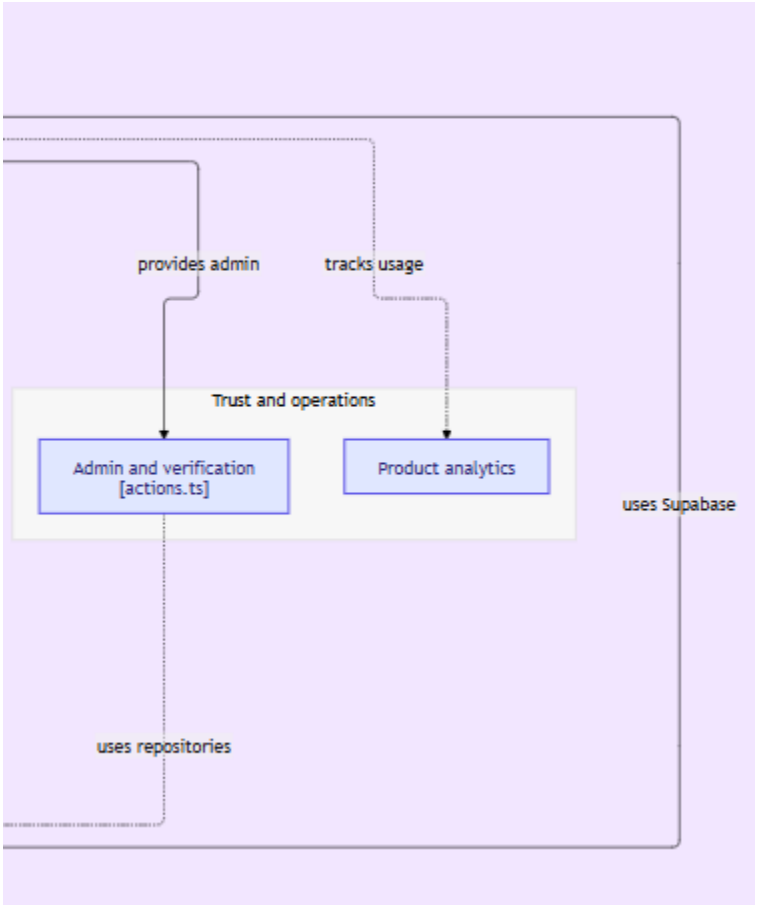
System Architecture

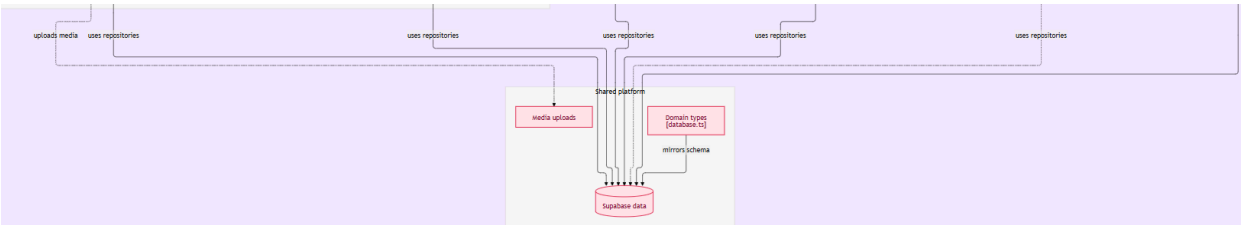


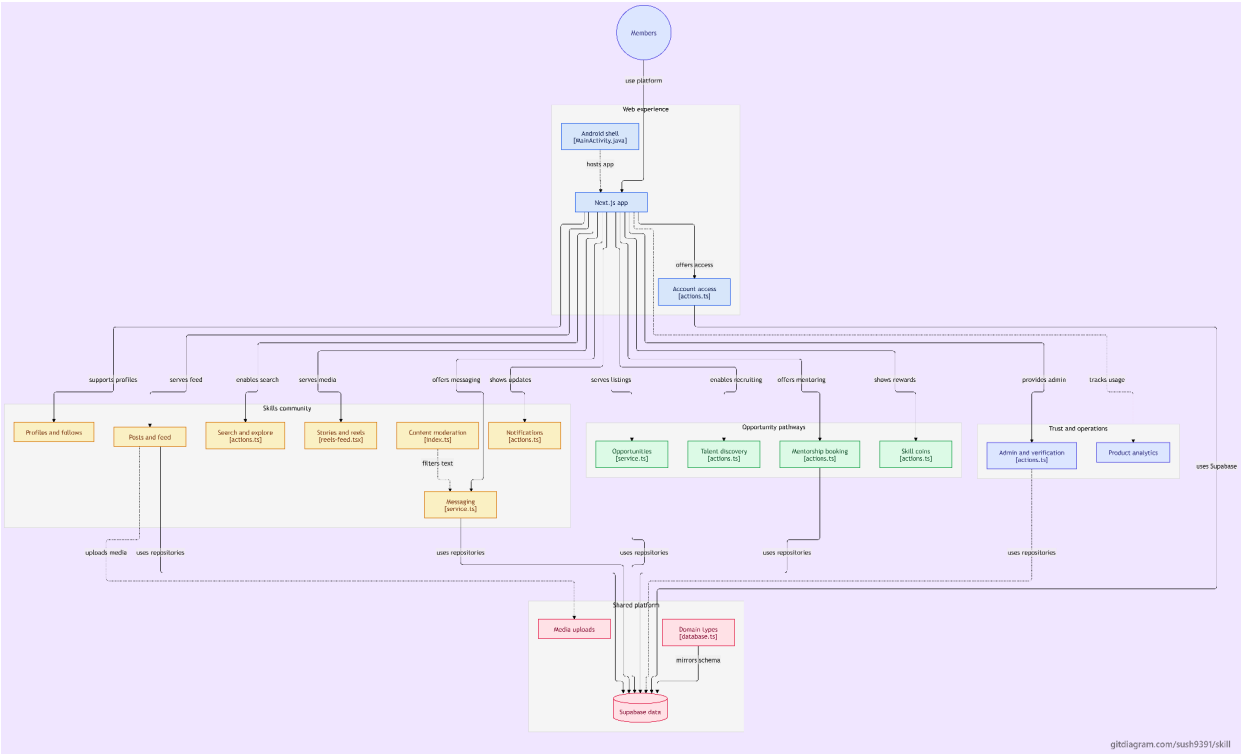

Members
use platform
Web experience
Android shell [MainActivity.java]
hosts app
Next.js app
offers access
Account access [actions.ts]


supports profiles
serves feed
enables search
serves media
offers messaging
shows updates
Skills community
Profiles and follows
Posts and feed
Search and explore [actions.ts]
Stories and reels [reels-feed.tsx]
Content moderation [index.ts]
Notifications [actions.ts]
filters text
Messaging [service.ts]
uploads media
uses repositories
uses repositories


serves listings
enables recruiting
offers mentoring
shows rewards
Opportunity pathways
Opportunities [service.ts]
Talent discovery [actions.ts]
Mentorship booking [actions.ts]
Skill coins [actions.ts]


provides admin
tracks usage
Trust and operations
Admin and verification [actions.ts]
Product analytics
uses Supabase
uses repositories


uploads media
uses repositories
uses repositories
uses repositories
uses repositories
uses repositories
Shared platform
Media uploads
Domain types [database.ts]
mirrors schema
supabase data



2. Monorepo Architecture

The project codebase is structured as an **npm workspace monorepo**, establishing a strict separation between client runtime shells and modular, reusable core packages.
The repository root is partitioned into three core directories:

- **apps/**: Contains the application entry points, including the Next.js web application root (`apps/web`) and the Capacitor Android platform shell (`apps/mobile`).
- **packages/**: Houses shared packages used across applications, including authentication (`auth`), database repositories and migrations (`database`), content moderation logic (`moderation`), TypeScript definitions (`types`), reusable UI components (`ui`), workspace configurations (`config`), and shared utilities (`utils`).
- **e2e/**: Holds the Playwright end-to-end integration test suite for automated browser testing across the platform.

Core Shared Packages

- **`packages/auth`**: Encapsulates login/signup workflows, session recovery, Zod schemas, OAuth strategies, and password reset flows.
- **`packages/database`**: Implements specialized repositories for domains (e.g., profiles, posts, messaging, notifications, gamification, mentorship).
- **`packages/moderation`**: Contains synchronous rule-based detectors for automated moderation checks prior to persistent storage.

3. Layered System Architecture & Data Flow

Layer Name	Technologies / Components	Role & Data Flow Description
1. Entry Point Layer	Web Browser, Android Native Wrapper (Capacitor)	Serves as the primary entry point for user interactions across web platforms and native mobile shells, routing client calls to appropriate endpoints.
2. User-Facing Layer	Next.js App Router, React Components, Tailwind CSS v4, Framer Motion	Renders interactive user interfaces, manages micro-animations, handles client-side state, and presents features like feeds, profiles, and messaging.
3. Admin Endpoints Layer	Moderator Panel, Admin Verifications, Service Management Routes	Exposes privileged system endpoints and administrative dashboards for content moderation, platform settings, user verification, and elevated system oversight.
4. Service Layer	Server Actions, Domain Feature Services, Zod Schemas, Supabase Auth	Processes business logic, validates payloads, enforces authentication/authorization, and orchestrates workflows for social feeds, jobs, mentorship, and messaging.

Layer Name	Technologies / Components	Role & Data Flow Description
5. Utility Layer	FFmpeg WebAssembly, Cloudinary, Storage Utilities, PostHog Analytics, Push Workers	Provides cross-cutting services including media processing, image optimization, file uploads, event tracking, and background notification delivery.
6. Database Layer	PostgreSQL via Supabase, Row Level Security (RLS) Policies	Persists system entities and domain records while enforcing data isolation, access rules, schema constraints, and transactional consistency.

Feature Module Isolation

Business logic resides inside dedicated domain folders (`features/`) containing self-contained components, actions, services, and hooks:

- `features/auth/`
- `features/posts/`
- `features/messaging/`
- `features/notifications/`
- `features/opportunities/`
- `features/mentorship/`
- `features/stories/`
- `features/reels/`
- `features/profile/`
- `features/reports/`
- `features/talent/`
- `features/gamification/`

4. Authentication & Native Deep Linking Architecture

Identity and access management rely on Supabase Auth, implementing Proof Key for Code Exchange (PKCE) OAuth 2.0 flows across web and native runtime environments.

Distinct URI routing strategies are enforced based on the executing platform client context.

4.1 Web Browser Callback Flow

Web environments process authentication callbacks using standard HTTP redirects managed by Next.js Server Route Handlers. Upon identity provider verification, Supabase issues session tokens and redirects the user client to the target endpoint via query parameter routing:

```
Browser ——> OAuth Provider ——> Supabase Auth ——>
/auth/callback?redirectTo=...
```

4.2 Native Android Deep Link Flow

Web and Android Platform Architecture

The platform leverages a unified cross-platform architecture where the Next.js web application codebase serves as the single source of truth for both web and mobile deployments. The web frontend is packaged and compiled into a native Android shell via Capacitor, enabling direct deployment to Android targets while sharing 100% of the UI components, business logic, state management stores, and service integrations.

By embedding the Next.js application runtime inside Capacitor's native webview container, platform updates, domain features, and UI enhancements are maintained in a single workspace without requiring parallel codebases or duplicated development effort across web and mobile teams.

Native Android runtimes execute OAuth exchanges via the system browser through custom URI scheme delegation. Control returns seamlessly to the Android native shell via deep link intent filters:

```
Android App ——> System Browser (OAuth) ——> Supabase ——> Custom URI
Scheme (teachgrow.skilltego.com://auth-callback) ——> Native Intent
Filter ——> App State Restored
```

Security Policy Note: *The custom URI callback scheme must remain identical across the OAuth Provider settings, the AndroidManifest.xml intent filters, and the Supabase Redirect Allowlist to eliminate authentication loop vectors.*

5. Data Persistence, Security & Migrations

Data isolation, multi-tenancy rules, and user privacy constraints are directly enforced within PostgreSQL using **Row Level Security (RLS)** policies managed through Supabase migration scripts.

5.1 Sequential Schema Migration Sequence

Database migrations are deployed in strict sequential order to guarantee dependency resolution across system domain tables:

1. `0001_init` — Core users and profiles
2. `0002_posts` — Social posts and interactions
3. `0003_messaging_notifications` — Realtime chat & system alerts
4. `0004_opportunities_dashboards` — Organizations, job postings, & applications
5. `0005_admin` — Escalated platform permissions
6. `0006_stories` — Short-form stories and interactive sticker elements
7. `0007_message_reactions` — Direct messaging reactions
8. `0008_private_accounts` through `0024_profile_rewards_lockdown` — Granular visibility controls, Skill Coins ledger, web push tokens, and profile lockdown mechanics.

Deployment Operational Requirement: *All automated deployment pipelines must execute the full migration sequence (0001–0024) in order when provisioning initial target environments.*

5.2 Security Model & Access Controls

- **Public Profiles**: Basic user bio information, verified skill badges, and public posts remain readable platform-wide by authenticated users.
- **Private Accounts**: User content, stories, and social feed activity are restricted exclusively to approved followers through automated RLS join queries.
- **Personal Skill Coins**: Wallet balances, transaction histories, and reward point allocations are restricted strictly to account owners and system execution contexts.

6. Core Subsystems Specification

6.1 Media Processing Pipeline

- **Video Optimization**: Uses FFmpeg WebAssembly (`apps/web/public/ffmpeg`) client-side for initial thumbnail generation and client-side compression before streaming.
- **Storage Dual-Path**: Supports both direct Cloudinary CDN uploads and Supabase Storage buckets with resumable upload capabilities for large media.

6.2 Stories & Short-Form Video (Reels)

- **Stories**: Temporal content with a strict 24-hour retention lifecycle, supported by interactive stickers (Polls, Quizzes, Questions, Emoji Sliders).
- **Reels**: Vertical continuous scroll video feed backed by an active-video Zustand store managing background prefetching and audio context.

6.3 Realtime Messaging & Service Worker Push

- **Authorization**: Direct message routes check user dynamic participation in conversations prior to fetching transcript payload.
- **Push Services**: Client Service Worker (`sw.js`) intercepts push events, parses incoming JSON payloads (or plain text fallback), and surfaces actionable notifications.

6.4 Opportunities & Mentorship

- **Opportunity Engine**: Dashboards for companies and academic institutions to manage jobs, internships, competitions, and scholarship listings.
- **Mentorship Platform**: Conditional profile extensions exposing scheduling slots, automated session booking, and student reviews.

6.5 Gamification System

- **Skill Coins Engine**: Users earn platform-specific Skill Coins by completing daily check-ins, authoring highly-rated educational content, completing mentorship sessions, and successfully referring new platform members.
- **Rewards & Redemption**: Supports ranking tiers, verifiable skill badges, and digital reward redemptions guarded by profile-level transaction locks to prevent double-spending and abuse.

6.6 Content Moderation

- **Rule-Based Filtering**: Provides synchronous pattern analysis checking for profanity, spam heuristics, phishing URLs, and unauthorized contact exposure prior to persistent storage in database tables.
- **Queue Handling**: Flagged posts and reported items are queued for review in an isolated moderator panel (`/moderation`). Unprivileged non-moderator accounts attempting to access this path receive an explicit 404 response for security through obscurity.

7. Developer Workflow, CLI Commands & CI/CD

7.1 Environment Prerequisites

- Node.js >= 20.0.0
- npm >= 10.0.0
- Active Supabase Project instance

7.2 Local Environment Setup

```
# Clone repository & install monorepo dependencies

npm install

# Environment configuration

cp apps/web/.env.example apps/web/.env.local
```

7.3 Monorepo CLI Task Reference

Action	Command	Scope
Development	`npm run dev`	Starts Next.js app on `http://localhost:3000`
Production Build	`npm run build`	Compiles workspace apps and packages
Unit Testing	`npm test`	Executes Vitest runner across monorepo
Watch Unit Tests	`npm run test:watch`	Interactive Vitest continuous testing
E2E Testing	`npm run test:e2e`	Triggers Playwright headless browser suite
Formatting	`npm run format:check`	Checks Prettier formatting compliance
Linting	`npm run lint`	ESLint static analysis checking

7.4 Pull Request Validation Sequence

Prior to submitting code changes or opening pull requests, developers must run the complete automated check suite locally to ensure compliance:

```
npm run lint && npm run format:check && npm test && npm run build && npm run test:e2e
```